

Introduction to



with Application to Bioinformatics

- Day 2


```
In [ ]: type(5)
```

```
In [ ]: a = 3.14  
a + 2
```

```
In [ ]: x = [1,5,3,7,8]
        y = ['a','b','c']
        print(type(y))
        print(x + y) # list can have heterogenous data types
```


Comments (Q 3)

3. Which of the following symbols can be used to write comments in your code?

#

You can also use docstring `"""` to comment out multiple lines of code, although it is not designed for that.

6. How do you find out if 5 is larger than 3 and the integer 4 is the same as the float 4? Fill in all the missing code.

```
5 > 3 and 4 == 4.0
```

```
5 > 3 and 4 == float(4.0)
```

```
In [ ]: i = 2  
        f = 5.46  
        li1 = [1,2,3,4]  
        li2 = [5,6,7,8]
```

```
In [ ]: i * f
```

```
In [ ]: li1 * i
```

Basic operations

Type	Operations
int	+ - / ** % // ...
float	+ - / * % // ...
string	+

```
In [ ]: i = 2  
        f = 5.46  
        li1 = [1,2,3,4]  
        li2 = [5,6,7,8]
```

```
In [ ]: i * f
```

```
In [ ]: li1 * i
```

```
In [ ]: li1 * f
```



```
In [ ]: li = [1,2,3,4,5,6,7,8]
        b = 5
        c = 10
        b in li
```

```
In [ ]: b not in li
        # not b in li # works as well, but not recommended
```

```
In [ ]: b < c or c == 1
```


Sequences and collections (Q 7-9)

7. How do you select the second element in the variable `mylist = [4,3,8,10]` ?

Lists (and strings) are an ORDERED collection of elements where every element can be access through an index.

```
value: 4 3 8 10  
index: 0 1 2 3
```

Sequences and collections (Q 7-9)

7. How do you select the second element in the variable `mylist = [4,3,8,10]` ?

Lists (and strings) are an ORDERED collection of elements where every element can be access through an index.

```
value: 4 3 8 10  
index: 0 1 2 3
```

```
mylist[1]
```



```
In [ ]: a = [1,2,3,4,5]
        b = ['a','b','c']
        c = 'a random string'

        c[2]
        c[1:4]
```

```
In [ ]: li1 = ['a','b','c']      # mutable  
        str1 = 'abc'           # immutable
```

```
In [ ]: li1[2] = 'B'  
        li1
```

```
In [ ]: str1[2] = 'B'
```

```
In [ ]: myTuple = (1,2,3,4,'a','b','c')  
myList = [1,2,3,4,'a','b','c']
```

```
In [ ]: for item in myTuple:  
        print(item)
```

```
In [ ]: for item in myList:  
        print(item)
```

```
In [ ]: myList[2] = 42  
myList
```

```
In [ ]: myTuple[2] = 42
```


If/ Else statements (Q 10)

10. How do you do to print 'Yes' if x is bigger than y?

```
if x > y:  
    print('Yes')
```

```
In [ ]: a = 3  
b = [1,2,3,4]  
if a in b:  
    print(str(a)+' is found in the list b')  
else:  
    print(str(a)+' is not in the list')
```


Break (10 min)

0/1

- For human, and many other organisms, there are two copies of genetic information, inherited from both parents.
- 0 means DNA at this position is the same as reference allele (A),
- 1 means has alternative allele, in this case AC

Genotype field

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(MiSeq), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

0/1

- For human, and many other organisms, there are two copies of genetic information, inherited from both parents.
- 0 means DNA at this position is the same as reference allele (A),
- 1 means has alternative allele, in this case AC

The notation of alternative allele is not always 1 : 0/2

- 2 means it is the second alternative allele, i.e. C in this case


```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(Mich), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

- Open the file and loop over lines (ignore lines with started with #)
- Identify lines where chromosome is equal to 5 and position is between 1,200,000 and 1,300,000
- Isolate the column that contains the genotype for sample1, e.g. 0/1:30:SM
- Extract only the genotypes (e.g. 0/1) from the column
- Check if the genotype contains any alternate alleles, i.e., if not 0/0 .
- Print any variants containing alternate alleles for this sample between specified region

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(MiSeq), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [1]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            print(line)
            break
# Next, find chromosome 5
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(Mich), B(BI)">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(Mich), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [ ]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            if chrom == '5':
                print(cols)
                break

# Next, find the correct region
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(MiSeq), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [1]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                print(cols)
                break
# Next, find the genotypes for sample1
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(UMich), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [ ]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                print(gt_field)
                break

# Next, extract the genotypes only
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(Mich), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [ ]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                print(gt)
                break

# Next, find in which positions sample1 has alternate alleles
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(Mich), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [ ]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF processing, this condi
                    print(gt)
#Next, print nicely
```

```
##fileformat=VCFv4.2
##source=ExampleVCF
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=CB,Number=1,Type=String,Description="Called by S(Sanger), M(MiSeq), B(BI)">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT Sample1 Sample2 Sample3
1 10177 rs367896725 A AC 50 PASS . GT:DP:CB 0/1:30:SM 0/0:28:SMB 0/1:32:MB
1 13110 rs569393146 G A,C,T 99 PASS . GT:DP:CB 0/2:60:SM 1/1:55:SMB 2/3:58:MB
```

```
In [ ]: vcf_file = '../downloads/genotypes_small.vcf'
with open(vcf_file, 'r', encoding='utf-8') as fh:
    for line in fh:
        if not line.startswith('#'):
            line = line.strip()
            cols = line.split('\t')
            chrom = cols[0]
            pos = int(cols[1])
            if chrom == '5' and (1200000 <= pos <= 1300000):
                gt_field = cols[9]
                gt = gt_field.split(':')[0]
                if gt != "0/0": # For robust VCF processing, this condi
                    ref = cols[3]
                    alt = cols[4]
                    info = chrom + ":" + str(pos) + "_" + ref + "-" + a
                    print(info)
```



```
In [ ]: print("this is your sequence")
```

```
In [ ]: sorted([5,3,135,31])
```

```
In [ ]: " I'm a line with trailing spaces ".strip()
```

```
In [ ]: " I'm a line with trailing spaces ".split()
```

```
In [ ]: li = [3, 1, 5, 43, 5]  
li.sort()  
print(li)
```


Built-in Functions				
abs ()	delattr ()	hash ()	memoryview ()	set ()
all ()	dict ()	help ()	min ()	setattr ()
any ()	dir ()	hex ()	next ()	slice ()
ascii ()	divmod ()	id ()	object ()	sorted ()
bin ()	enumerate ()	input ()	oct ()	staticmethod ()
bool ()	eval ()	int ()	open ()	str ()
breakpoint ()	exec ()	isinstance ()	ord ()	sum ()
bytearray ()	filter ()	issubclass ()	pow ()	super ()
bytes ()	float ()	iter ()	print ()	tuple ()
callable ()	format ()	len ()	property ()	type ()
chr ()	frozenset ()	list ()	range ()	vars ()
classmethod ()	getattr ()	locals ()	repr ()	zip ()
compile ()	globals ()	map ()	reversed ()	__import__ ()
complex ()	hasattr ()	max ()	round ()	

```
In [ ]: help(max)
```

```
In [ ]: a = 5  
help([1,2,3])
```

```
In [ ]: for i in range(5):  
        print(i)
```


range

```
In [ ]: for i in range(5):  
        print(i)
```

```
In [ ]: # range(start, stop, step)  
        # generate a series of odd number  
        for i in range(1, 10, 2):  
            print(i)
```

range

```
In [ ]: for i in range(5):  
        print(i)
```

```
In [ ]: # range(start, stop, step)  
        # generate a series of odd number  
        for i in range(1, 10, 2):  
            print(i)
```

```
In [ ]: # negative step  
        for i in range(10, 1, -2):  
            print(i)
```

```
In [ ]: li = [5, 4, 1, 3, 10, 13, 13]
        print(sorted(li))
```

sorted

```
In [ ]: li = [5, 4, 1, 3, 10, 13, 13]  
print(sorted(li))
```

```
In [ ]: print(sorted(li, reverse=True))
```

sorted

```
In [ ]: li = [5, 4, 1, 3, 10, 13, 13]
        print(sorted(li))
```

```
In [ ]: print(sorted(li, reverse=True))
```

```
In [ ]: # sorted does not change the content in place
        li = [5, 4, 1, 3, 10, 13, 13]
        print(sorted(li))
        print(li)
        sorted_li = sorted(li)
        print(li)
```

```
In [ ]: dir(str)
```

```
In [ ]: a = "This is an ongoing course".strip().split().index('an')  
dir(a)
```



```
In [1]: a = ' split a "string into a list" '  
a.split()
```


str.split()

```
In [1]: a = ' split a "string into a list" '  
a.split()
```

```
In [1]: a.split(maxsplit=2)
```

```
In [1]: ''.join(["ATGC", "CCCCC", "AAATG", "CTTTGTA"]) # convert a list to stri
```



```
In [ ]: a.pop()  
a
```

```
In [ ]: a.pop(1)  
a
```

```
In [ ]: a.reverse()  
a
```

```
In [ ]: a.remove(5)  
a
```

```
In [ ]: a.insert(0, 13)  
a
```

```
In [ ]: ### Tuples are immutable  
a = (2, 3, 4, 15)  
a.pop()
```


#	Votes	Rating	Year	Runtime	URL	Genres	Title
126807		8.5	1957	5280	https://images-na.ssl-images...	Drama,War	Paths of Glory
71379		8.2	1925	4320	https://images-na.ssl-images...	Adventure,Comedy,Drama,Family	The Gold

```
In [ ]: hit = [0.0, ""]
with open("../downloads/250.imdb", "r", encoding="utf-8") as fh:
    for line in fh:
        if not line.startswith("#"):
            cols = line.strip().split('|')
            rating = float(cols[1])
            movie_title = cols[6].strip()
            if (rating > hit[0]):
                hit = [rating, movie_title]

print(hit)
```

- Open file
- Initiate counter to keep track of highest rating and movie. Start counter at 0
- Loop over all lines not starting with '#'
- Strip and split the lines into a list
- Save the element containing the rating from the list into a variable
- If current rating is higher than the rating in the counter, replace counter value with rating and movie
- Close file
- Print counter

```
# Votes | Rating | Year | Runtime | URL | Genres | Title
126807| 8.5|1957|5280|https://images-na.ssl-images....|Drama,War|Paths of Glory
71379| 8.2|1925|4320|https://images-na.ssl-images....|Adventure,Comedy,Drama,Family|The Gold
```

```
In [ ]: fh = open('../downloads/250.imdb', 'r', encoding='utf-8')
best = [0, ''] # here we save the rating and which movie
for line in fh:
    if not line.startswith('#'):
        cols = line.strip().split('|')
        rating = float(cols[1].strip())
        if rating > best[0]: # if the rating is higher than p
            best = [rating, cols[6]]
fh.close()
print(best)
```